

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 3

По дисциплине: «Современные методы защиты компьютерных систем»

Тема: «Разработка модулей ядра операционной системы для динамического перехвата системных вызовов с использованием технологии eBPF»

Выполнил:

Студент 4 курса

Группы ИИ-25

Трошин И.Д.

Проверила:

Хацкевич А.С.

Цель работы: практическое освоение методов разработки программ расширенного фильтра пакетов Беркли (eBPF) на языке C для мониторинга событий уровня ядра, динамического перехвата системных вызовов (kprobes/tracerepoints) и выявления аномального поведения процессов в операционной системе Linux с использованием фреймворка BCC (BPF Compiler Collection).

Вариант 14

Вариант	Целевой системный вызов (или tracerpoint)	Условие аномального поведения (Фильтр ядра)
14	sys_enter_fchownat	Попытка смены владельца любого файла на суперпользователя (UID = 0)

Ход работы

Задание 1 (Базовый перехват запуска процессов):

1. Листинг eBPF-программы (C-код для ядра и Python-обертка) для перехвата вызова `execve`.

```
#!/usr/bin/python3
from bcc import BPF
import time

# =====
# 1. Kernel Space - код на C
# =====
bpf_text = """
#include <uapi/linux/ptrace.h>
#include <linux/sched.h>

// Структура для передачи данных из ядра в User Space
struct data_t {
    u32 pid;
    u32 uid;
    char comm[TASK_COMM_LEN];
};

// Объявление кольцевого буфера (Perf Ring Buffer) с именем 'events'
BPF_PERF_OUTPUT(events);

// Функция, которая прикрепляется к tracerpoint sched_process_exec
TRACEPOINT_PROBE(sched, sched_process_exec)
{
    struct data_t data = {};

    // ЗАДАНИЕ 1:
    // 1. Извлекаем PID
    data.pid = bpf_get_current_pid_tgid() >> 32;

    // 2. Извлекаем UID
    data.uid = bpf_get_current_uid_gid();

    // 3. Извлекаем имя процесса
    bpf_get_current_comm(
        &data.comm,
        sizeof(data.comm)
    );

    // 4. Отправляем структуру data в пространство пользователя
    events.perf_submit(
        args,
        &data,
        sizeof(data)
    );

    return 0;
}
"""

# =====
# 2. Инициализация eBPF
# =====
# Компилируем C-код и загружаем его в виртуальную машину eBPF в ядре
b = BPF(text=bpf_text)

print("[*] Мониторинг запущен. Ожидание запуска процессов...")
print("[*] Нажмите Ctrl+C для выхода.")
```

```
# =====
# 3. User Space - обработка событий в Python
# =====
def print_event(cpu, data, size):
    # Конвертируем сырые байты из ядра в структуру Python
    event = b["events"].event(data)

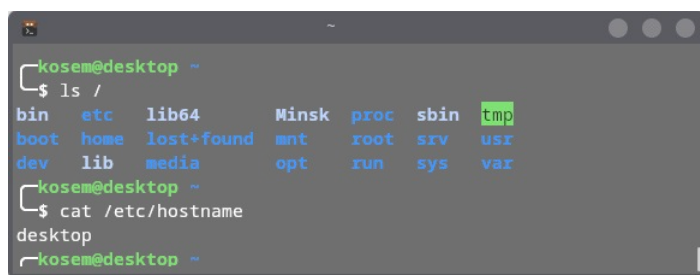
    comm = event.comm.decode(
        "utf-8",
        "replace"
    ).rstrip("\x00")

    print(
        f"PID: {event.pid} | "
        f"UID: {event.uid} | "
        f"Process: {comm}"
    )

# Подписываем функцию print_event на события из кольцевого буфера 'events'
b["events"].open_perf_buffer(print_event)

# Бесконечный цикл опроса буфера
while True:
    try:
        b.perf_buffer_poll()
    except KeyboardInterrupt:
        print("\n[*] Завершение работы монитора.")
        exit()
```

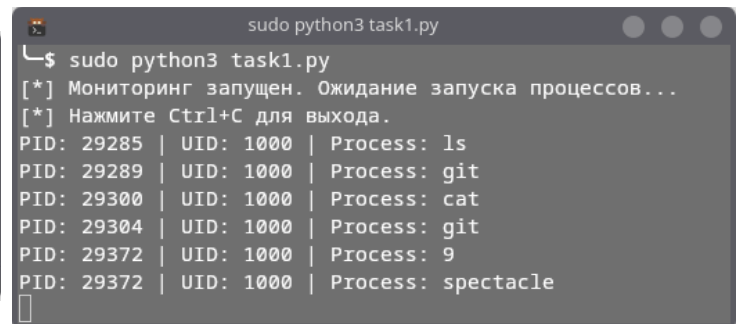
2. Скриншот вывода терминала, демонстрирующий перехват запуска команд ls или cat, с отображением PID и имени процесса (comm).



```
kosem@desktop ~
$ ls /
bin  etc  lib64  Minsk  proc  sbin  tmp
boot home lost+found mnt  root  srv  usr
dev  lib  media  opt   run   sys  var

kosem@desktop ~
$ cat /etc/hostname
desktop

kosem@desktop ~
```



```
sudo python3 task1.py
$ sudo python3 task1.py
[*] Мониторинг запущен. Ожидание запуска процессов...
[*] Нажмите Ctrl+C для выхода.
PID: 29285 | UID: 1000 | Process: ls
PID: 29289 | UID: 1000 | Process: git
PID: 29300 | UID: 1000 | Process: cat
PID: 29304 | UID: 1000 | Process: git
PID: 29372 | UID: 1000 | Process: 9
PID: 29372 | UID: 1000 | Process: spectacle
```

Задание 2 (Динамический анализ аномального поведения):

1. Сформулированное условие из индивидуального варианта.

Попытка смены владельца любого файла на суперпользователя (UID = 0)

2. Листинг С-кода зонда. Обязательно выделите строками логику фильтрации, которая отсекает легитимные вызовы прямо в ядре и вызывает функцию отправки данных в кольцевой буфер только при совпадении с условием.

```
#include <uapi/linux/ptrace.h>
#include <linux/sched.h>

struct data_t {
    u32 pid;
    u32 uid;

    u32 target_uid;
    u32 target_gid;

    char comm[TASK_COMM_LEN];
    char filename[256];
};

BPF_PERF_OUTPUT(events);

TRACEPOINT_PROBE(syscalls, sys_enter_fchownat)
{
    struct data_t data = {};
```

```

// Фильтрация выполняется непосредственно в ядре.
// Нас интересуют только попытки установить владельца UID = 0.

if (args->user != 0)
    return 0;

data.pid = bpf_get_current_pid_tgid() >> 32;
data.uid = bpf_get_current_uid_gid();

data.target_uid = args->user;
data.target_gid = args->group;

bpf_get_current_comm(
    &data.comm,
    sizeof(data.comm)
);

bpf_probe_read_user_str(
    &data.filename,
    sizeof(data.filename),
    args->filename
);

events.perf_submit(
    args,
    &data,
    sizeof(data)
);

return 0;
}

```

3. Скриншот двух терминалов: в первом — запуск вашего Python-монитора, во втором — генерация целевой аномалии (например, `cat /etc/shadow` или `chmod 777 file.txt`). В первом терминале должен быть зафиксирован алерт.

```

kosem@desktop ~
$ sudo chown 1000 file.txt
kosem@desktop ~
$ sudo chown 0 file.txt
kosem@desktop ~
$ 

```

```

kosem@desktop ~/Desktop/СМЗКС/lab3
$ sudo python3 task2.py
[*] Мониторинг fchownat запущен.
[*] Аномалия: попытка установить UID владельца = 0.
[*] Нажмите Ctrl+C для выхода.
[ALERT] PID: 33318 | UID: 0 | Process: chown | File: file.txt | Target UID: 0 | Target GID: 4294967295

```

Задание 3 (Интеграция с пространством пользователя и аудит):

1. Фрагмент измененного Python-скрипта, реализующий запись алертов в файл `security_audit.log`.

```

message = (
    f"{timestamp} | "
    f"PID: {event.pid} | "
    f"UID: {event.uid} | "
    f"Process: {comm} | "
    f"File: {filename} | "
    f"Target UID: {event.target_uid} | "
    f"Target GID: {event.target_gid}"
)

print("[ALERT] " + message)
with open("security_audit.log", "a") as log_file:

```

```
log_file.write(message + "\n")
```

2. Текстовый вывод или скриншот содержимого файла security_audit.log с зафиксированной аномалией (должны присутствовать Timestamp, PID, UID и суть события).

```
└─$ cat security_audit.log
```

```
2026-10-06 12:13:40 | PID: 20266 | UID: 0 | Process: chown | File: /tmp/ebpf_test.txt | Target UID: 0 | Target GID: 4294967295
```

```
2026-10-06 12:13:42 | PID: 20295 | UID: 0 | Process: chown | File: /tmp/ebpf_test.txt | Target UID: 0 | Target GID: 4294967295
```

Задание 4 (Проектирование профилей защиты):

1. Развернутый ответ на пункт 1: техническое обоснование различий между Seccomp-BPF (механизм ограничения доступа) и традиционными eBPF kprobes (механизм трассировки), с объяснением причин, по которым прерывание работы системного вызова из kprobe может привести к нестабильности ОС.

Seccomp-BPF — это механизм ограничения доступа процесса к системным вызовам. Программа seccomp подключается к фильтрации системных вызовов и получает информацию о вызываемом syscall и его аргументах. На основании этой информации она может разрешить или запретить выполнение системного вызова. Например, можно настроить процесс так, чтобы вызов `execve()` был запрещён, а при попытке его выполнить ядро завершало процесс или возвращало ошибку. Таким образом, seccomp используется прежде всего как механизм **предотвращения нежелательных действий** и ограничения возможностей конкретного процесса.

Традиционный eBPF kprobe, напротив, предназначен прежде всего для **наблюдения и трассировки работы ядра**. kprobe позволяет установить точку наблюдения на функцию ядра или системный вызов. При достижении этой точки выполняется eBPF-программа, которая может получить информацию о происходящем событии, отфильтровать её и передать данные в пространство пользователя. Например, в данной лабораторной работе через kprobe или tracerpoint можно отслеживать запуск процессов или вызовы `fchownat()` и фиксировать попытки установить владельца файла с UID 0.

2. Развернутый ответ на пункт 2: академическое описание уязвимости TOCTOU в контексте перехвата аргументов-указателей (pointers) системных вызовов в eBPF и предложения по безопасному копированию памяти (`bpf_probe_read_user_str`).

Уязвимость TOCTOU (Time-of-Check to Time-of-Use — «проверка во время одного момента, использование во время другого») возникает, когда программа сначала проверяет некоторое значение, а затем использует его, предполагая, что за прошедшее время значение осталось неизменным. В контексте мониторинга системных вызовов с помощью eBPF проблема особенно актуальна для аргументов системных вызовов, представляющих собой указатели на память пользовательского пространства.

Например, системный вызов `fchownat()` получает указатель на строку с именем файла:

```
fchownat(..., const char *filename, ...);
```

При перехвате системного вызова eBPF-программа получает не саму строку, а **адрес памяти**, по которому эта строка находится. Этот адрес относится к пользовательскому адресному пространству и потенциально может измениться или стать недействительным после момента проверки.

Таким образом, данные, которые монитор считал при проверке, могут отличаться от данных, которые фактически использовались позже. Злоумышленник потенциально может попытаться воспользоваться этой разницей, например подменив имя файла между проверкой и использованием.

Поэтому нельзя рассматривать пользовательский указатель как обычную безопасную строку. eBPF-программа должна обращаться к пользовательской памяти через специальные безопасные механизмы ядра.

В контексте eBPF хорошей практикой является получение необходимого значения из пользовательской памяти один раз, копирование его в локальную структуру события и дальнейшая работа уже с этой копией:

```
bpf_probe_read_user_str(
    &data.filename,
    sizeof(data.filename),
    args->filename
);
```

Вывод: освоил методы разработки программ расширенного фильтра пакетов Беркли (eBPF) на языке C для мониторинга событий уровня ядра, динамического перехвата системных вызовов (kprobes/tracerepoints) и выявления аномального поведения процессов в операционной системе Linux с использованием фреймворка BCC (BPF Compiler Collection).